

In-Depth Analysis of Homomorphic Encryption Libraries

Itai Zloclzower Jacob Shemesh Tomer Ovadya Yoni Glickstein

Confidential Computing Course
Ben-Gurion University of the Negev
July 2026

Abstract

Homomorphic encryption lets a server compute on data it cannot read. It closes the one gap that ordinary encryption leaves open: data is protected at rest and in transit, but not while it is being used. The price is speed, and this project measures how high that price is. We implement the same operations three times — in plain Python, in TenSEAL, and in OpenFHE — and measure runtime, memory, ciphertext size and accuracy. Every figure is a mean over 1000 repetitions after a discarded warm-up. Our main finding is that there is no single overhead number. In TenSEAL, encrypted addition costs $153\times$ a plaintext addition, but summing one encrypted vector costs $32,832\times$ its plaintext equivalent. What separates them is whether the operation has to move data between ciphertext slots. Re-running OpenFHE at TenSEAL’s parameters then cuts its multiplication penalty from $6.89\times$ to $3.16\times$, so much of the gap between the libraries is really a gap between configurations. Applied to a synthetic healthcare score over 1000 records, the encrypted computation finishes in 214ms, matches the plaintext answer to five significant figures, and costs $2,865\times$ plaintext rather than the $13,450\times$ measured on isolated operations — because a full ciphertext amortises a cost that CKKS charges per ciphertext, not per value.

1 Introduction

Encryption protects data at rest and in transit, but not while it is being used. To compute on encrypted data, a normal system has to decrypt it first, and at that moment the plaintext is exposed to whatever machine is doing the work. Homomorphic encryption (HE) removes that step: operations on ciphertexts produce ciphertexts that decrypt to the correct answer, so a cloud provider can process data it never sees.

The idea is old, but usable libraries are recent. That turns a theoretical capability into a practical decision: if a hospital or a bank wanted to outsource a computation over sensitive records today, what would it cost, and which library should it choose? Answering that needs measurement. Published complexity results say little about what a particular library does with a particular operation on a particular machine.

This project measures it. We compare two CKKS libraries, **TenSEAL** and **OpenFHE**, against an unencrypted baseline over a small set of operations common in data analysis, and report runtime, memory, ciphertext size, accuracy, and the practical problems we met while building them.

Research question. *What is the practical overhead of performing basic computations using two different homomorphic encryption implementations, compared to equivalent plaintext computation?*

The report follows the measurements in the order they led to one another. Section 2 explains the scheme and the libraries, Section 3 what we built and the three experiments we ran, and Section 4 how we measured them. Section 5 gives the core result: what each operation costs. That result raises an immediate question — are we measuring the libraries, or the settings we

gave them? — which Section 6 answers. Section 7 then puts everything together in a healthcare example, and Sections 8–9 cover usability and limitations.

2 Background

2.1 Homomorphic encryption and CKKS

Both libraries implement **CKKS** [1], a scheme for approximate arithmetic on real numbers. Approximate is the important word: decrypting a CKKS result gives the correct value plus a small error, in exchange for much better performance than exact schemes on real-valued data. For statistics over measurements that trade is usually fine, and Section 5.4 shows the error we actually saw.

Four properties of CKKS shape every measurement in this report.

- **Ring dimension.** Ciphertexts are polynomials. The degree of that polynomial — the ring dimension — sets the security level and drives the cost of every operation. A bigger ring is more secure and uniformly slower.
- **Slot packing.** One ciphertext holds many values in parallel “slots”, and one operation acts on all of them at once. The cost depends on the ring dimension, not on how many slots hold real data.
- **Multiplicative depth.** Each multiplication uses up one level from a budget fixed at setup. Running past the budget does not slow the computation down; it makes the answer wrong.
- **Rotations.** Slots cannot be read individually. Summing across them means rotating the ciphertext and adding, about $\log_2(\text{slots})$ times, and every rotation needs an expensive key switch. This is why summation costs so much more than addition.

2.2 The two libraries

TenSEAL is a Python library built on Microsoft SEAL. It presents encrypted vectors as ordinary Python objects that support `+`, `*` and `.sum()`, and handles relinearization and rescaling internally. The programmer gives it the modulus chain directly.

OpenFHE is a C++ library with Python bindings, maintained by an academic consortium. It exposes lower-level calls (`EvalAdd`, `EvalMult`, `EvalSum`) and needs evaluation keys to be generated explicitly. The programmer gives it a multiplicative depth and a scaling size, and the library works out the ring dimension that satisfies them at the requested security level.

That difference matters more than it looks. Given what we considered comparable settings, the two libraries ended up at different ring dimensions — 8192 for TenSEAL and 16384 for OpenFHE — which we return to in Section 6.

2.3 Related work

Comparing HE libraries is not new. Sathya et al. [4] survey the available libraries and their trade-offs. Faneela et al. [5] benchmark SEAL against OpenFHE across BGV and CKKS on two operating systems and conclude that OpenFHE performs better. Takeshita et al. [6] profile four CKKS implementations and discuss directly how hard it is to compare libraries whose architectures differ. Viand et al. [7] survey the compiler layer above these libraries.

Our ranking differs from Faneela et al., which is one reason Section 6 exists: we found the ranking depends on the parameters each library is given. We claim no new method over the work above. This is a course project, and its goal is a clear practical measurement of two libraries we implemented ourselves.

3 Implementation

We implemented the same seven operations three times: in plain Python as a baseline, in TenSEAL, and in OpenFHE.

Operation	What it does
Key generation	Build the crypto context and generate the keys the library needs
Encryption	Encode a vector of numbers into a ciphertext
Addition	Element-wise sum of two encrypted vectors
Multiplication	Element-wise product of two encrypted vectors
Summation	Sum of all elements inside one encrypted vector
Average	Summation, then multiplication by $1/n$
Dot product	Element-wise multiplication, then summation
Decryption	Recover the plaintext result

Table 1: The implemented operations. All except key generation and encryption have a direct plaintext counterpart, which is the baseline.

The two libraries need different code for the same computation. Encrypted addition in TenSEAL is `enc + enc`; in OpenFHE it is `cc.EvalAdd(ct, ct)`. Summation is `enc.sum()` against `cc.EvalSum(ct, n)`, and the OpenFHE version needs summation keys generated beforehand.

One incompatibility showed up immediately and affected everything after it: **OpenFHE requires the batch size to be a power of two**, while TenSEAL accepts any length. Every input has to be padded up to the next power of two. For simple operations that is harmless, but Section 7 shows a case where the padding *value* has to be chosen carefully.

Each operation is timed on freshly encrypted inputs rather than on the output of the previous repetition, so no measurement inherits accumulated noise or a consumed level. All three implementations share one measurement harness, so the plaintext, TenSEAL and OpenFHE columns are produced by the same timing and memory code and differ only in the operation being called.

3.1 The three experiments

The measurements are organised as three experiments. The first answers the research question directly; the two after it exist because the one before left something unexplained.

Experiment	Question it answers
1 Basic operations (§5)	What does each encrypted operation cost, against the same operation in plaintext?
2 Matched parameters (§6)	Is the gap between the libraries, or between the settings they chose by default?
3 Healthcare use case (§7)	What happens when the operations are combined into one complete workload?

Table 2: The three experiments and what each one is for.

Experiment 2 is one of the optional extensions named in the proposal, different parameter settings. Experiment 3 is the healthcare use case, implemented rather than only discussed. The encrypted dot product, also optional in the proposal, is measured as part of Experiment 1.

4 Experimental Setup

4.1 What we measure, and how

Runtime. Each operation is timed on its own with `time.perf_counter` around a single call. Encryption and decryption are timed separately from the operations between them, so the cost of getting data in and out is never folded into the cost of computing on it.

Repetitions and warm-up. One timing of an HE operation is not reproducible. Every figure in this report is the **mean of 1000 timed repetitions after 50 warm-up calls that are discarded**. The warm-up matters because the first calls into either library pay costs that are not part of normal operation: loading the native library, building lookup tables on first use,

and faulting in fresh memory. Counting them would inflate the mean and make it depend on how many repetitions were run. Garbage collection is off inside the timed loop, so an unrelated collection is not charged to the operation. Every repetition starts from the same freshly encrypted inputs, so no repetition inherits noise or a consumed level from the one before it.

Memory. We report two figures, because one is not enough. Python’s `tracemalloc` sees only Python-level allocations, and both libraries keep ciphertext coefficients in native C++ memory that Python never learns about. So we also allocate many ciphertexts, hold them, and measure how much the process resident set grows. Memory is measured in a separate pass from timing, because `tracemalloc` hooks every allocation and distorts short measurements.

Ciphertext size. Each library’s own serialization routine, in bytes, compared against the plaintext vector stored as 8-byte doubles.

Accuracy. Every encrypted result is compared against the plaintext computation of the same values, and we report the largest absolute error. Each benchmark checks this error against a loose bound before reporting any timing, so a run with wrong parameters fails visibly instead of producing plausible but meaningless numbers.

4.2 Machine and configuration

Measurements ran on a compute node reserved exclusively for the job, so nothing else shared the machine. We fixed the thread count to one (`OMP_NUM_THREADS=1`): both libraries use multi-threaded native code, and on a shared machine their measured cost would depend on how many cores each happened to get, which is not a property of the library.

CPU	Intel Xeon E5-2680 v3 @ 2.50 GHz, single-threaded, exclusive node
OS / Python	Rocky Linux 9.7 / Python 3.8.20
Libraries	TenSEAL 0.3.15, OpenFHE 1.4.1, NumPy 1.24.4
TenSEAL parameters	<code>poly_modulus_degree</code> = 8192, coefficient moduli [60, 40, 40, 60], scale 2^{40}
OpenFHE parameters	multiplicative depth 3, scaling modulus 50 bits, 128-bit security $\Rightarrow$ ring 16384
Input	The vector [1.0, 2.0, 3.0, 4.0, 5.0] unless stated otherwise

Table 3: Experimental configuration. Each parameter set is that library’s natural way of expressing comparable settings; Section 6 examines what the difference costs.

Absolute timings depend on this hardware, so the *ratios* between the three columns are the result that transfers, not the millisecond values.

5 Results: The Cost of Basic Operations

5.1 Runtime

Operation	Plaintext (ms)	TenSEAL (ms)	OpenFHE (ms)	TenSEAL overhead	OpenFHE overhead
Encrypt	—	5.4914	15.1101	—	—
Add	0.000629	0.0961	0.3869	153×	615×
Multiply	0.000550	4.7413	32.4827	8,616×	59,025×
Sum	0.000320	10.5082	76.1546	32,832×	237,943×
Average	0.000349	11.7961	81.3563	33,794×	233,072×
Dot product	0.000993	11.0764	96.6155	11,152×	97,272×
Decrypt	—	1.4029	22.6702	—	—
Total (5 comparable ops)	0.002841	38.2180	286.9961	13,450×	101,003×

Table 4: Mean runtime over 1000 repetitions. The total covers the five operations that exist in all three columns; encryption and decryption have no plaintext counterpart and are left out of it.

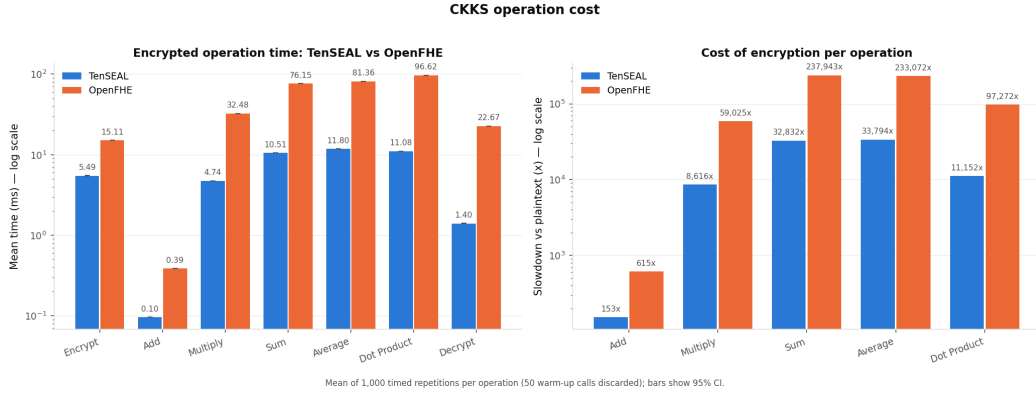


Figure 1: Runtime per operation on a logarithmic scale, and the slowdown against plaintext. Both libraries span two orders of magnitude across operations.

Three things stand out.

There is no single overhead figure. Inside TenSEAL alone the cost runs from 153× for addition to 32,832× for summation. A claim like “HE is ten thousand times slower” is therefore not usable: the answer depends on which operation dominates the workload.

Addition is nearly free; operations that need rotations are not. Encrypted addition takes 96 microseconds, a fiftieth of a multiplication and a hundredth of a summation. The reason is structural. Addition works on ciphertext coefficients directly. Summation, average and dot product have to move data between slots, and every rotation carries a key switch. This one mechanism explains the order of every row in the table, and it is the most useful thing we measured: a computation built from additions and constant multiplications will be far cheaper than one that has to aggregate across slots.

TenSEAL is faster here, on every operation. It leads by 2.75× on encryption and 6.85× on multiplication. Section 6 tests whether that reflects the libraries or the parameters we gave them.

5.2 Key generation

Key generation happens once per session rather than once per operation, so we timed it separately, over 50 repetitions.

	TenSEAL	OpenFHE
Context and key generation	216.180 ms	208.308 ms

This is the one measurement where OpenFHE leads, and the two are within 4% of each other. The more useful observation is the scale: TenSEAL spends 216 ms building keys and 38 ms doing all five encrypted operations. Setup costs almost six times the computation, so a service that builds a fresh context per request would spend most of its time generating keys rather than doing the work it was asked to do.

5.3 Memory and ciphertext size

Measure	TenSEAL	OpenFHE
Python-visible memory (<code>tracemalloc</code>), encryption	0.59 KB	0.42 KB
Resident memory held per ciphertext	359,465 B	1,258,455 B
Serialized ciphertext	334,428 B	1,312,467 B
Plaintext vector (5 doubles)	40 B	

Table 5: Memory for the 5-element vector. The first row measures Python objects; the second and third measure the ciphertext itself.

The first row is why we measured memory two ways. On its own, `tracemalloc` suggests a ciphertext costs under a kilobyte, which is wrong by a factor of about 600: it is measuring the Python wrapper, not the encrypted data behind it. The resident-set figure and the serialized size agree with each other to within 7%, which is reassuring, since they are independent measurements of the same object.

The practical consequence is the last row. Five numbers take 40 bytes in the clear and 334 KB encrypted under TenSEAL — an expansion of about $8,400\times$. For a cloud deployment this, not RAM, is the binding constraint, because it is what has to cross the network. Section 7 shows the ratio improving sharply once the ciphertext slots are actually used.

We also measured the evaluation keys, which the server needs before it can compute at all. At the configuration used in Section 7 they are **179.7 MB** for TenSEAL and **69.2 MB** for OpenFHE, against roughly 5 MB of encrypted data. They are sent once per context rather than per query, but they are by far the largest object involved.

5.4 Accuracy

Operation	TenSEAL	OpenFHE
Add	1.57×10^{-9}	4.09×10^{-14}
Multiply	3.35×10^{-6}	1.96×10^{-12}
Sum	4.60×10^{-7}	7.11×10^{-15}
Average	3.10×10^{-7}	5.20×10^{-14}
Dot product	5.19×10^{-6}	3.27×10^{-13}

Table 6: Largest absolute error against the plaintext result.

Both libraries are accurate enough for what CKKS is meant for: an error of 10^{-6} on values of order 10 is far below the precision of any measurement one would realistically encrypt. Multiplication gives the largest error in both, which follows from the scheme — multiplying two ciphertexts multiplies their noise, and the rescaling afterwards drops low-order bits.

OpenFHE is several orders of magnitude more precise throughout. Part of that is its larger scaling factor, 2^{50} against TenSEAL’s 2^{40} , which keeps about ten more bits of every value. Section 6 shows that is not the whole story.

6 The Libraries, or the Parameters?

Section 5.1 found TenSEAL faster on every operation and Section 5.4 found OpenFHE more precise. Both comparisons share a weakness: the two libraries were not running equivalent parameters. OpenFHE had chosen ring dimension 16384 against TenSEAL’s 8192, so it was doing roughly twice the polynomial work per operation, and it was using a 2^{50} scaling factor against 2^{40} , so it was keeping ten more bits of every value. We cannot credit either result to the library until those differences are removed.

OpenFHE lets the ring dimension be set explicitly, so we re-ran it pinned to 8192 with a 2^{40} scale, matching TenSEAL. We also varied OpenFHE’s *scaling technique*, an internal option controlling how it manages the scaling factor between multiplications, which TenSEAL does not expose at all.

OpenFHE configuration	Encr	Add	Mult	Sum	Dot	Decr	Geo. mean
Own defaults (ring 16384, 2^{50})	2.52	4.02	6.89	7.24	8.70	16.28	6.44
Matched, same scaling technique	1.20	2.12	3.16	3.25	3.97	7.83	3.05
Matched, fastest scaling technique	1.02	1.77	1.93	2.41	3.09	6.80	2.37

Table 7: OpenFHE runtime as a multiple of TenSEAL’s, per operation. Lower is closer to TenSEAL.

Matching the parameters closes most of the gap. Multiplication drops from $6.89\times$ to $3.16\times$,

encryption becomes almost identical at $1.20\times$, and across all six operations the geometric mean falls from $6.44\times$ to $3.05\times$. Separating the two changes for multiplication, the ring dimension accounts for a factor of 2.18 and the scaling technique for a further 1.64.

Two qualifications go with this result. Decryption is still $7.8\times$ slower even matched, so the gap narrows everywhere but closes nowhere. And pinning the ring dimension overrides OpenFHE’s own security check: asked to pick a ring for these parameters at 128-bit security, OpenFHE chooses 16384 and refuses 8192. The matched configuration therefore buys comparability by giving up security, which is fine for an experiment isolating one variable but would not be acceptable in deployment.

What we take from this is that **a benchmark of two HE libraries at their default settings measures the defaults as much as the libraries**. It also suggests why Faneela et al. [5] rank OpenFHE above SEAL where we rank TenSEAL above OpenFHE: at default settings, the two are not doing equivalent work.

7 Use Case: Privacy-Preserving Healthcare Analytics

Everything so far concerns isolated operations. A real workload combines them, and combining them raises problems that single operations do not. To see that, we implemented the motivating scenario from our proposal: a hospital wants a cloud provider to compute a score for every patient and the average over the cohort, without exposing any individual record.

The score. We define a **synthetic risk score** over five measurements per patient. *This score is built for demonstration only. It is not a medical model and has no clinical meaning.* Each feature is first normalised to roughly $[0, 1]$ against a fixed public range, $x_{\text{norm}} = (x - \min)/(\max - \min)$, using age 18–90, systolic blood pressure 90–180, BMI 18–40, cholesterol 120–280 and glucose 70–200. The score is then

$$\begin{aligned} \text{score} = 100 \cdot (& 0.15 a + 0.20 b + 0.15 m + 0.15 c + 0.15 g \\ & + 0.10 b g + 0.05 m c + 0.05 b^2), \end{aligned}$$

where a, b, m, c, g are the normalised age, blood pressure, BMI, cholesterol and glucose. The eight weights sum to 1, so a patient at the top of every range scores 100.

The shape was chosen to stress the library, not to model medicine. The first five terms are a weighted average and need only multiplication by public constants. The last three are what make it a useful test: two interaction terms and one square, each multiplying one ciphertext by another. That is the operation that consumes multiplicative depth, and it is why this computation cannot run under the parameters of Section 5.

The data. A synthetic cohort of 1000 patients, generated from a fixed seed. Age is drawn uniformly across its range; the four clinical measures are drawn from normal distributions centred on plausible values and then clipped to the ranges above, which keeps every normalised value inside $[0, 1]$. The resulting scores have mean 35.63 and standard deviation 8.45.

Encrypting it. We encrypt one *feature* per ciphertext, one patient per slot, giving five ciphertexts of 1000 values each. A single homomorphic multiplication then advances all 1000 patients at once. The alternative — one ciphertext per patient — would multiply the number of operations by a thousand.

The padding rule from Section 3 gets delicate here. Vectors are padded from 1000 to 1024 slots, and we fill the padding with *each feature’s minimum*, not with zero. A minimum normalises to exactly 0, so padded slots contribute nothing and drop out of the cohort sum. Zero-padding would instead give those slots a normalised value of $-\min/(\max - \min)$, which is not zero, and would quietly corrupt the average. That is exactly the kind of bug that produces a plausible wrong answer, so we cover it with a unit test.

Depth. Normalisation, the interaction terms, the weights and the cohort mean use four levels in total. Neither library can run that at the Section 5 settings: TenSEAL fails with `scale out of bounds`, and a longer modulus chain does not fit at ring 8192, so the ring has to double to 16384. OpenFHE evaluates the score at depth 3 but fails on the cohort mean and needs depth 4. Adding two interaction terms and a square to a weighted average was enough to force new cryptographic parameters and double the ring dimension — in CKKS, the shape of a computation matters more than the amount of data in it.

7.1 Results

Stage	Runs at	Plaintext (ms)	TenSEAL (ms)	OpenFHE (ms)
Encrypt 5 features	Hospital	—	76.784	84.650
Evaluate the score	Cloud	—	89.249	163.181
Cohort mean	Cloud	—	46.104	146.054
Decrypt the scores	Hospital	—	2.186	17.457
Total		0.0748	214.323	411.341
Overhead		1×	2,865×	5,499×
Per patient		0.000075	0.2143	0.4113

Table 8: The full scenario over 1000 patients. The plaintext column is the single computation producing the same output: a score per patient and the cohort mean.

	TenSEAL	OpenFHE
Largest absolute error (score points)	4.881×10^{-4}	1.976×10^{-10}
Cohort mean, encrypted	35.627391	35.626948
Cohort mean, plaintext	35.626948	

Table 9: Accuracy of the encrypted score. The worst per-patient error is 5×10^{-4} points on a 0–100 scale.

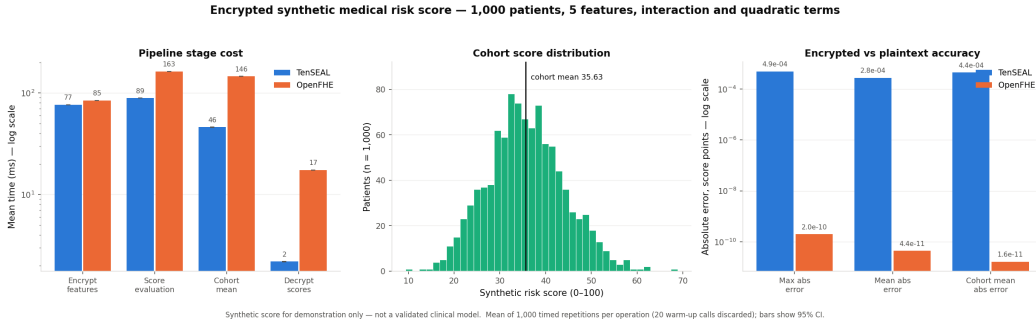


Figure 2: Cost per stage, the distribution of scores across the synthetic cohort, and the accuracy of the encrypted result.

The whole scenario takes **214 ms** with TenSEAL and 411 ms with OpenFHE, of which the cloud does 135 ms and 309 ms. Per patient that is 0.21 and 0.41 ms, and the encrypted cohort mean matches the plaintext value to five significant figures.

The overhead is worth comparing against Section 5: 2,865× here against 13,450× for the isolated operations. Nothing got faster. The ciphertexts got fuller — 1024 slots carrying real data instead of 5 — and a CKKS operation costs what the ring dimension says it costs, however many slots actually hold data. Measuring HE on toy inputs therefore overstates its cost.

The same effect shows up in data volume. Each encrypted feature is about 1 MB, so the hospital uploads roughly 5 MB for a cohort that takes 40 KB in the clear — an expansion of 132×, against the 8,400× measured on the 5-element vector. The evaluation keys, at 179.7 MB, are still larger than the data.

Two caveats. Encryption and decryption are done by the hospital, so 79 ms of the 214 ms is work the client does rather than work it avoids; for a computation this small, outsourcing would not pay for itself, and the scenario is better read as a stand-in for workloads where it would. And our benchmark runs client and server in one process, so it measures the computation faithfully but not the network transfer or key management a real deployment would also need.

8 Ease of Implementation and Usability

Aspect	TenSEAL	OpenFHE
Installation	<code>pip install</code> , any platform	Linux only, one Python version
Writing a computation	Python operators on encrypted vectors	Explicit method calls
Key management	Handled by the context	Evaluation keys generated by hand
Parameters	Modulus chain given directly	Depth given, ring dimension derived
Vector length	Any	Powers of two only
Error messages	Python exceptions	C++ errors through the bindings
Benchmark code	92 lines	111 lines

TenSEAL was clearly easier to work with. Encrypted code looks like the plaintext code next to it, and the context handles keys without being asked. OpenFHE asks more of the programmer: evaluation keys generated explicitly, batch sizes rounded to powers of two, and several concepts named that TenSEAL keeps internal.

That verbosity is not only a cost. OpenFHE makes you declare the multiplicative depth up front, so exceeding it fails immediately and says so. TenSEAL finds the same problem later and reports `scale out of bounds`, whose connection to the length of the modulus chain is not obvious to a newcomer — we spent noticeably longer diagnosing that failure in TenSEAL than the equivalent in OpenFHE. OpenFHE also exposes controls, such as the scaling technique in Section 6, that turn out to matter and that TenSEAL does not offer at all.

9 Practical Limitations

Met during development.

1. **OpenFHE installs only on Linux, and only for one Python version.** Its wheel ships a binary built for CPython 3.8; installing it under 3.9 or later appears to work and then fails at import. This fixed the Python version for the whole project.
2. **Power-of-two batch sizes.** Every input has to be padded, and Section 7 shows the padding value is not always free to choose.
3. **Multiplicative depth has to be planned in advance.** Adding two interaction terms to a weighted average forced new parameters and doubled the ring dimension. Asking for more depth than needed costs just as much: our first attempt requested depth 5, which works but pushes the ring to 32768 and slows every operation for no benefit.
4. **Measuring memory is not straightforward.** The obvious tool reports a figure about 600 times too small, because the data lives outside Python.

Limitations of this study.

1. Measurements come from one machine, one CPU generation and a single thread. The ratios between columns transfer better than the absolute times.
2. The plaintext baseline is ordinary Python. A NumPy or C baseline would be faster, so our overhead figures are upper bounds relative to optimised plaintext.
3. Each configuration was measured in one process run, so variation between processes is not captured.
4. Neither library was tested with bootstrapping, so all results concern computations that fit inside a fixed level budget.

10 Conclusions

The research question asked what basic computations cost under homomorphic encryption compared with plaintext. The measurements give a specific answer and, more usefully, explain what drives it.

- 1. The overhead is a range, not a number.** Encrypted operations cost between $153\times$ and $32,832\times$ their plaintext equivalents in TenSEAL, and between $615\times$ and $237,943\times$ in OpenFHE. Any single quoted figure hides a factor of two hundred.
- 2. What an operation costs depends on whether it needs rotations.** Addition and constant multiplication work on ciphertexts directly and are cheap. Summation, average and dot product have to move data between slots, and every rotation carries a key switch. Structuring a computation to avoid aggregation across slots is the single most effective optimisation available.
- 3. Much of the difference between the libraries was a difference in parameters.** Re-running OpenFHE at TenSEAL’s ring dimension and scale cut its multiplication penalty from $6.89\times$ to $3.16\times$ and made encryption nearly identical. Comparing two HE libraries at their defaults is partly a comparison of the defaults.
- 4. Real computations are cheaper than isolated operations suggest, so ciphertexts should be filled.** The healthcare scenario — 1000 records, interaction terms, a cohort aggregate — ran at $2,865\times$ plaintext, a fifth of the isolated benchmark’s overhead, and agreed with the plaintext result to five significant figures. Nothing became faster; the ciphertexts simply carried 1024 values instead of 5, and CKKS charges per ciphertext rather than per value. The largest data cost was not the ciphertexts but the evaluation keys: 179.7 MB against 5 MB of encrypted data.

Homomorphic encryption is usable today for the kind of computation examined here, as long as the workload is batched, structured to avoid unnecessary rotations, and given parameters chosen deliberately rather than by default. The cost is real, but it is dominated by choices the programmer controls.

Two further experiments are included in the project repository rather than here: a sweep of the pipeline over input sizes from 5 to 500, which measures the filling effect above directly, and a measurement of the cost of OpenFHE’s noise-flooding decryption mode.

References

- [1] J. H. Cheon, A. Kim, M. Kim, Y. Song. Homomorphic encryption for arithmetic of approximate numbers. *ASIACRYPT*, 2017.
- [2] A. Al Badawi et al. OpenFHE: open-source fully homomorphic encryption library. *WAHC*, 2022.
- [3] A. Benaissa, B. Retiat, B. Cebere, A. E. Belfedhal. TenSEAL: a library for encrypted tensor operations using homomorphic encryption. 2021.
- [4] S. S. Sathya, P. Vepakomma, R. Raskar, R. Ramachandra, S. Bhattacharya. A review of homomorphic encryption libraries for secure computation. arXiv:1812.02428, 2018.
- [5] Faneela, J. Ahmad, B. Ghaleb, S. U. Jan, W. J. Buchanan. Cross-platform benchmarking of the FHE libraries: novel insights into SEAL and OpenFHE. arXiv:2503.11216, 2025.
- [6] J. Takeshita, N. Koirala, C. McKechney, T. Jung. HEPProfiler: an in-depth profiler of approximate homomorphic encryption libraries. *Journal of Cryptographic Engineering*, 2025.
- [7] A. Viand, P. Jattke, A. Hithnawi. SoK: fully homomorphic encryption compilers. *IEEE Symposium on Security and Privacy*, 2021.
- [8] M. Albrecht et al. Homomorphic Encryption Security Standard. HomomorphicEncryption.org, 2018.